Class 8

Sunday, June 21, 2026 4:36 PM

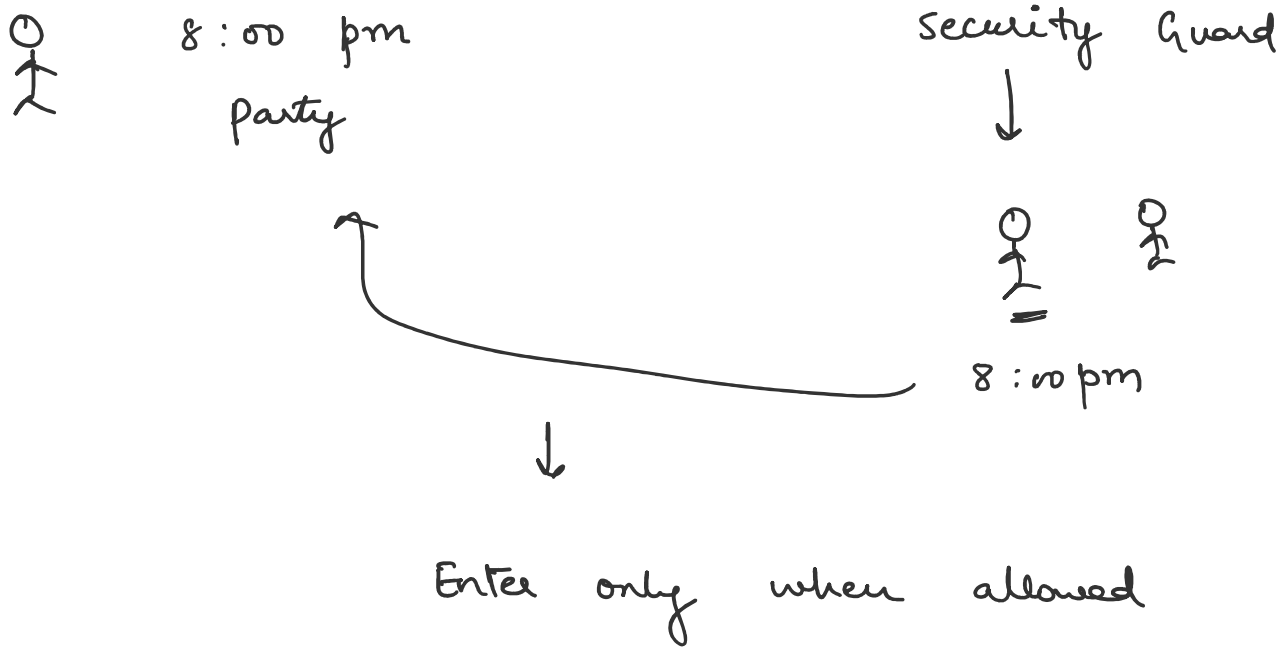
Singleton

instance = new Singleton() ✓✓

Lazy loading

↓

On demand



instance == null → security check

↓

instance = new Singleton()

Lazy loading issue

↓

Not thread safe

↓

To solve this we have 2 sol^n's.

Sol^n 1 - using synchronized.

Washroom

↓

check if available or not

↓

Washroom available

↓

You get phone call

↓

Another person comes

↓

He uses the washroom

↓

Go inside

↓

Both inside → Invalid

↓

Race around condⁿ.

↓

Solⁿ - Syn block

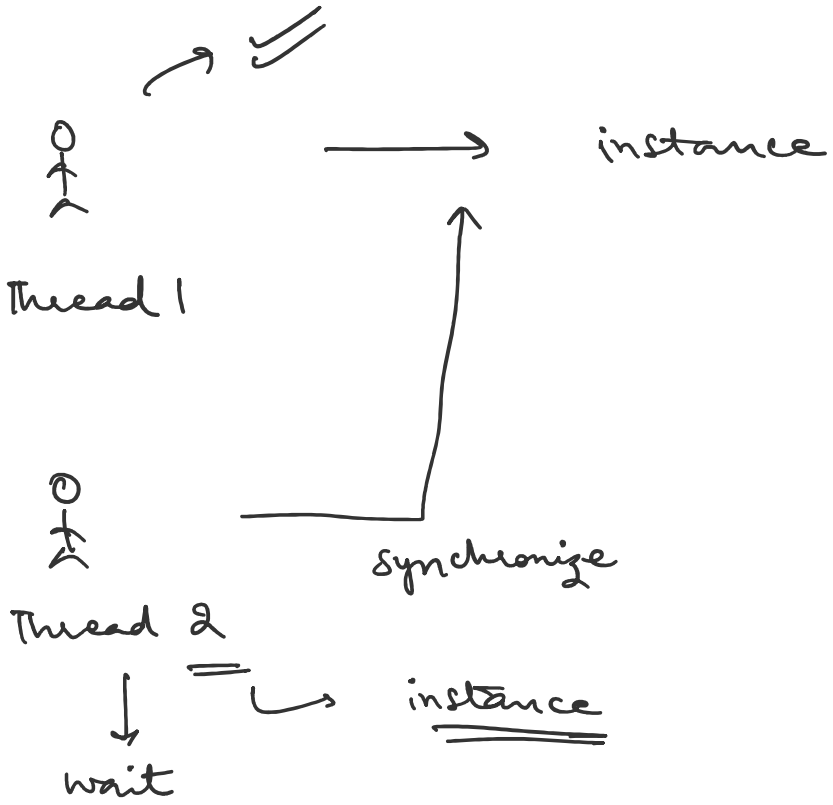
public static synchronized Singleton

getInstance ()

{

}

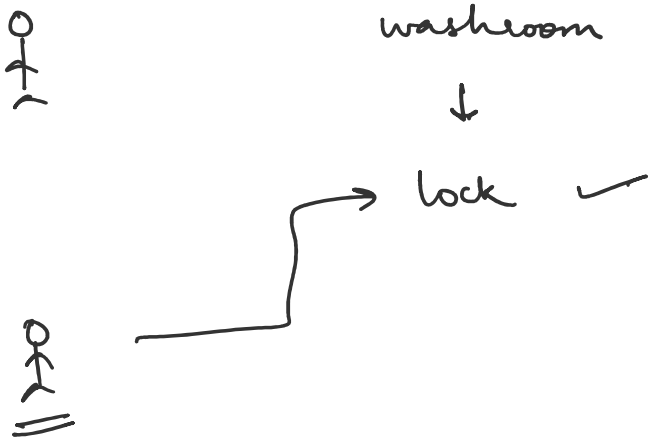
only one thread can enter at a time.



using synchronized on fn will slow

down the performance because even after singleton is created then also every call has to wait.

Soln 2 - Using Double locking



private static volatile Singleton instance;

checks for stale data

All threads will see latest value of object

of object

```
public static Singleton getInstance ()
 {

        Ist lock.

        if (instance == null)
        {                                  IInd lock
                                          ↗
            synchronized (Singleton.class)
            {
                if (instance == = null)
                {
                    instance = new Singleton()
                }
            }
          return instance.
    }
```

only first thread will enter

synchronized block.

Lazy Loading

- Object is created **on demand**
- **Saves memory** if the object is never used
- Allows **passing parameters** to the constructor during creation
- Simple and efficient for **single-threaded environments**

In programming, **lazy loading** means the object is only created when it's actually needed. This saves memory and is more efficient.

How It Works

Instead of creating the instance upfront, we delay creation:

```
if (instance == null) {
    instance = new Singleton(…);
}
```

The object is created only the **first time getInstance() is called**.

Implementation

```
public class Singleton {
   private Singleton(String createdBy) {
      System.out.println("Instance created by " + createdBy);
   }
```

```
    private static Singleton instance;

    public static Singleton getInstance(String createdBy) {
        if (instance == null) {
            instance = new Singleton(createdBy);
        }
        return instance;
    }
}



import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) {
        ExecutorService exe = Executors.newCachedThreadPool();
        Runnable task1 = () -> Singleton.getInstance("Achint");
        Runnable task2 = () -> Singleton.getInstance("Anjali");

        exe.execute(task1);
        exe.execute(task2);
        exe.shutdown();
    }
}

/* *
Output:
    Instance created by Achint
    Instance created by Anjali
* */
```

Advantages

- Saves memory
- Object created on demand
- Supports constructor parameters

Disadvantages

- Not thread safe
- Breaks singleton in multi-threaded environments

Problem in Multithreading

Lazy loading **fails in multi-threaded environments**.

If two threads execute this at the same time:

```
if (instance == null)
```

Both may see instance == null and proceed to create objects.

Washroom Analogy

Without Lock:

1. You go to washroom.
2. Check if anyone is inside → No.
3. You receive a phone call.
4. Another person comes.
5. Checks → No one inside.
6. Goes inside.
7. You finish call and also go inside.

Result:
Two people entered.
Exactly the same thing happens in Lazy Singleton.

Race Condition Problem

Suppose:
if (*instance* == null)
Two threads execute simultaneously.

Thread A:
instance == null

Thread B:
instance == null

Both create objects.

Result:
Object 1
Object 2
Singleton is broken.

This resulted in:

- **Multiple instances created**
- Singleton contract is broken

This happened because:

- The check (instance == null) is **not atomic**
- No synchronization between threads
- Threads execute independently

Thread Safe Lazy Singleton

Thread safe lazy loading fixes the problem of multiple threads creating many objects.

Solution 1 -

To fix the race condition in lazy loading, we use the synchronized keyword so that **only one thread can create the instance at a time**.

```
public static synchronized Singleton getInstance(String createdBy) {
    if (instance == null) {
        instance = new Singleton(createdBy);
    }
    return instance;
}
```

Here **getInstance()** method is marked synchronized, so **only one thread** can enter this method at a time. It checks: "Is the object already created?"

- If **no**, it creates it.
- If **yes**, it just returns the existing one.

Without synchronized, **multiple threads** might create **multiple instances** like earlier we saw and this **violates** the Singleton rule. With synchronized, only one thread can enter at a time, so **only one object is ever created**.

```
public class Singleton {
    private Singleton(String createdBy) {
        System.out.println("Instance created by " + createdBy);
    }

    private static Singleton instance;

    public static synchronized Singleton getInstance(String
createdBy) {
        if (instance == null) {
            instance = new Singleton(createdBy);
        }
        return instance;
    }
}
```

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) {
        ExecutorService exe = Executors.newCachedThreadPool();
        Runnable task1 = () -> {
            Singleton.getInstance("Achint");
        };

        Runnable task2 = () -> {
            Singleton.getInstance("Anjali");
        };

        exe.execute(task1);
        exe.execute(task2);
        exe.shutdown();
    }
```

```
}

/* *
Output:
    Instance created by Anjali
* */
```

Advantages

- Thread safety guaranteed
- Lazy loading achieved
- Supports constructor parameters

Disadvantages

- Every call acquires lock
- Slow performance because of locking overhead.
- Unnecessary synchronization that is not required once the instance variable is initialized.

Using synchronized on the entire method **slows down** access slightly because even **after** the Singleton is created, every call to **getInstance()** has to wait for the lock.

That's why Solution 2 (**double-checked locking**) is used in performance-critical apps.

Solution 2 -

Double-checked locking is a way to use synchronized **only when needed** and avoid locking **after** the object is created.

“Double-check” means first check **without locking** and then, inside the synchronized block, check **again**.

Real-Life Analogy -

Imagine you are waiting to get a ticket at a counter. Let’s walk through it step-by-step:

Imagine:

1. 100 people stand outside ticket counter.
2. Board says: **Ticket NOT Ready**
3. Everyone checks board - **Fast operation.**
   1. One person sees "Not Ready" and enters queue.
   2. Others wait outside.
   3. First person prepares ticket.
   4. Board changes to: **Ticket Ready**
   5. Everyone now directly takes ticket.

So only the **first thread** goes through the slow path. All others take the **fast path**, just reading the variable. This is **Double Checked Locking.**

Here we need to do 2 things -

1. Mark instance as volatile, without volatile, one thread might **see stale data** and still think the ticket is "Not Ready", even after it is. That's why we mark the instance like this:

```
private static volatile Singleton instance;
```

It ensures that **once created**, all threads see the **latest value** of the

object.

2. getInstance() method will have lock using
synchronized (Singleton.class)

This line is saying: "*Only one thread can enter this block of code at a time, and we are using the Singleton.class object as the lock.*"

The synchronized keyword in Java is used to make a **block of code thread-safe**. It ensures that **only one thread** can **execute the code inside** the block **at a time**. This prevents **race conditions** — situations where multiple threads try to change the same data at the same time, causing bugs.

In Java, every class (like Singleton) has a special class-level object called its **Class object**. Singleton.class refers to this **Class object**. It's a unique object — there's only **one per class**, no matter how many times the class is loaded or used. So we use Singleton.class as a **lock object**.

```
public class Singleton {
    private Singleton(String createdBy) {
        System.out.println("Instance created by " + createdBy);
    }

    private static volatile Singleton instance;

    public static Singleton getInstance(String createdBy) {
        if (instance == null) {
            synchronized (Singleton.class) {   // This is the locking part
                if (instance == null) {
                    instance = new Singleton(createdBy);
                }
```

```
            }
        }

        return instance;
    }
}



import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) {
        ExecutorService exe = Executors.newCachedThreadPool();
        Runnable task1 = () -> {
            Singleton.getInstance("Achint");
        };

        Runnable task2 = () -> {
            Singleton.getInstance("Anjali");
        };

        exe.execute(task1);
        exe.execute(task2);
        exe.shutdown();
    }
}

/* *
Output:
    Instance created by Achint
* */
```

Step-by-step:

1. Thread 1 checks instance == null → true.
2. Thread 1 enters the synchronized block → it locks on Singleton.class.
3. While Thread 1 is inside, Thread 2 also comes.
4. Thread 2 tries to enter the block, but it **must wait** because the lock is already taken by Thread 1.
5. Thread 1 creates the instance and exits.
6. Thread 2 now enters, but sees instance != null and skips creating another object.

Pros

- Thread safety is guaranteed.
- Lazy loading is achieved.
- Better performance
- Synchronization overhead is minimal and applies only to the first few threads when the variable is null.

Cons

- More complex
- Extra if condition.

Enum Singleton

- Simplest and safest Singleton implementation in Java.

- **enum** guarantees only one instance is created, and it handles thread safety internally.
- The Java Compiler and JVM take care of everything — no need to write synchronized code.

```
enum Singleton {
    INSTANCE;   // only object of this enum

    public static Singleton getInstance() {
        return Singleton.INSTANCE;
    }

    public void showMessage(String message) {
        System.out.println("Printing : " + message);
    }
}


public class Main {
    public static void main(String[] args) {
        Singleton singleton1 = Singleton.getInstance();
        singleton1.showMessage("Hello from Enum Singleton
instance 1!"); // Output: Printing : Hello from Enum Singleton
instance 1!

        Singleton singleton2 = Singleton.getInstance();
        singleton2.showMessage("Hello from Enum Singleton
instance 2!");    // Output: Printing : Hello from Enum Singleton
instance 2!
    }
}
```

Pros :

1. Thread Safe by Default

No synchronized. No volatile.

2. Safe from Serialization

Deserialization still returns same object.

3. Safe from Reflection

Reflection cannot create another instance.

4. Easy to Write

```
enum Singleton {
    INSTANCE
}
```

5. Guarantees One Object

Only one instance exists.

Cons :

1. No Lazy Loading

Object is created when enum loads.

2. Cannot Extend Another Class

Enums cannot inherit from classes.

3. Doesn't Work Well with DI Frameworks

Examples:

- Spring
- Guice

4. Difficult to Mock
Testing becomes harder.

5. Cannot Pass Dynamic Parameters
Constructor arguments cannot be supplied dynamically.

Why Singleton Is Often Avoided

1. Makes Unit Testing Difficult
Suppose:

PaymentService
    |
    V
DatabaseSingleton

During testing we may want **MockDatabase** instead of real database.

But code directly calls: **DatabaseSingleton.getInstance()** which makes mocking difficult.

2. Global State Problem
Singleton behaves like a global variable. Any class can modify it. This makes debugging harder.

3. Tight Coupling
Classes become dependent on: **Singleton.getInstance()** instead of abstractions.

4. Violates Dependency Injection Principles

Modern frameworks prefer constructor injection instead of global objects.